Practical: 1

Data Frames and Basic Data Pre-processing.

A. Reading data from CSV and JSON into a DataFrame

```
import pandas as pd

# 1) Read data from CSV file

data_csv = {
"Student_ID": [101, 102, 103, 104, 105],
"Name": ["Alice", "Bob", "Charlie", "David", "Eva"],
"Math": [85, 78, 92, 88, 76],
"Science": [90, 74, 89, 95, 80],
"English": [88, 69, 94, 91, 72]
}

df_csv = pd.DataFrame(data_csv)

print("Dataset loaded from CSV:")
print(df_csv.head())
print(df_csv.info())          # gives info on data types and non-null counts


# 2) Read data from JSON file
data_json = [
{"Product_ID": 1, "Product_Name": "Laptop", "Price": 75000, "Stock": 10},
{"Product_ID": 2, "Product_Name": "Mobile", "Price": 30000, "Stock": 25},
{"Product_ID": 3, "Product_Name": "Tablet", "Price": 20000, "Stock": 15},
{"Product_ID": 4, "Product_Name": "Headphones", "Price": 3000, "Stock": 50},
{"Product_ID": 5, "Product_Name": "Smartwatch", "Price": 15000, "Stock": 20}
]
df_json = pd.DataFrame(data_json)

print("\nDataset loaded from JSON:")
print(df_json.head())
print(df_json.info())
```

B. Performing Basic Data Pre-Processing

```
import pandas as pd
import seaborn as sns


# Load Titanic dataset
df = sns.load_dataset("titanic")

print("Dataset loaded from CSV:")
print(df.head())
print("\nDataset Information:")
print(df.info())


#Basic Data Pre-processing: Missing Values
print("\nBasic Data Pre-processing: Missing Values")

print("\nNumber of missing values per column:")
print(df.isnull().sum())

# 1) Replace missing values with 0
print("\n1) Replace missing values with 0")
df_filled = df.fillna(0)
print("\nAfter fillna(0) — first 10 rows:")
print(df_filled.head(10))

# Save the pre-processed data to a new Excel file
df_filled.to_excel('C:/Users/PC/Downloads/titanic_replaced.xlsx', index=False)

print("\nPre-processed data saved as 'titanic_replaced.xlsx'")

# 2) Drop rows with missing values
print("\n2) Drop rows with missing values")
df_dropped = df.dropna()
print("\nAfter dropna() — first 10 rows:")
print(df_dropped.head(10))

print("\nDataset info after dropna():")
print(df_dropped.info())

# Save the pre-processed data to a new Excel file
df_dropped.to_excel('C:/Users/PC/Downloads/titanic_droped.xlsx', index=False)

print("\nPre-processed data saved as 'titanic_droped.xlsx'")


#Manipulate and Transform Data
print("\nManipulate and Transform Data")
print("Original Data:")
print(df.head())

# 1) Filtering: Passengers who Survived
```

```
survived = df[df['Survived'] == 1]
print("\nPassengers who survived:")
print(survived)

# 2) Sorting: Sort by Fare in descending order
sorted_fare  = df.sort_values(by='Fare', ascending=False)
print("\nSorted by Fare (Descending):")
print(sorted_fare )

# 3) Grouping: Average Age by Passenger Class
grouped_class = df.groupby('Pclass')['Age'].mean()
print("\nAverage Age per Passenger Class:")
print(grouped_class)
```

Practical: 2

Feature Scaling and Dummification

Source Code :

```
import pandas as pd
from sklearn.preprocessing import StandardScaler, MinMaxScaler, LabelEncoder

# Create a sample dataset
data = pd.DataFrame({
    "Age": [25, 30, 35, 40, 28, 32, 38, 45],
    "Income": [50000, 60000, 70000, 80000, 55000, 65000, 75000, 90000],
    "Education": ["Bachelor", "Master", "PhD", "Master", "Bachelor", "Master", "PhD",
"Master"],
    "Marital_Status": ["Single", "Married", "Single", "Married", "Single", "Married", "Single",
"Married"]
})
print(data)

# Apply Standardization (Z-Score Scaling)
data[['Age_scaled', 'Income_scaled']] = StandardScaler().fit_transform(data[['Age',
'Income']])
print("\nData after Standardization:\n", data[['Age', 'Income', 'Age_scaled',
'Income_scaled']])

# Apply Normalization (Min-Max Scaling)
data[['Age_normalized', 'Income_normalized']] = MinMaxScaler().fit_transform(data[['Age',
'Income']])
print("\nData after Normalization:\n", data[['Age', 'Income', 'Age_normalized',
'Income_normalized']])

#Label Encoding (simple integer encoding)
le = LabelEncoder()
data['Education_code'] = le.fit_transform(data['Education'])
data['Marital_Status_code'] = le.fit_transform(data['Marital_Status'])
print("\nData with Encoded Categorical Variables:\n", data)


# Perform Feature Dummification for Categorical Variables
data_dummified = pd.get_dummies(data, columns=["Education", "Marital_Status"],
drop_first=True)
print("\nData after Dummification:\n", data_dummified)
```

Output:

Practical: 3

Hypothesis Testing

Source Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats
import seaborn as sns
import warnings

# ============================
# PART 1 : TWO SAMPLE T-TEST
# ============================

# Create two random samples
np.random.seed(42)
sample1 = np.random.normal(10, 2, 30)
sample2 = np.random.normal(12, 2, 30)

# Perform T-test
t_statistic, p_value = stats.ttest_ind(sample1, sample2)
alpha = 0.05

print("T-Test Results")
print("T-statistic:", round(t_statistic, 2))
print("P-value:", round(p_value, 4))

if p_value < alpha:
    print("Result: Reject Null Hypothesis (Means are different)")
else:
    print("Result: Fail to Reject Null Hypothesis (Means are similar)")


# Plot histograms
plt.hist(sample1, alpha=0.5, label="Sample 1")
plt.hist(sample2, alpha=0.5, label="Sample 2")
plt.title("Sample Distributions")
plt.legend()
plt.show()


# ============================
# PART 2 : CHI-SQUARE TEST
# ============================

# Load dataset
df = sns.load_dataset('mpg')

# Create categories using binning
df['horsepower_cat'] = pd.cut(df['horsepower'], bins=[0,75,150,240],
```

```
labels=['Low','Medium','High'])
df['year_cat'] = pd.cut(df['model_year'], bins=[69,72,74,84], labels=['T1','T2','T3'])

# Contingency Table
table = pd.crosstab(df['horsepower_cat'], df['year_cat'])
print("\nContingency Table:\n", table)

# Chi-square test
chi2, p, dof, expected = stats.chi2_contingency(table)

print("\nChi-Square Results")
print("Chi-square:", round(chi2,2))
print("P-value:", round(p,4))

if p < alpha:
    print("Result: Significant relationship exists")
else:
    print("Result: No significant relationship")
```

Output:

Practical: 4

ANOVA (Analysis of Variance)

Source Code:

```
import numpy as np
import pandas as pd
import scipy.stats as stats
import statsmodels.api as sm
from statsmodels.formula.api import ols
from statsmodels.stats.multicomp import pairwise_tukeyhsd

# ----------------------------
# Sample data (three groups)
# ----------------------------
group1 = [23, 20, 25, 22, 21]
group2 = [27, 30, 29, 31, 28]
group3 = [35, 37, 36, 34, 38]

# Combine data into a DataFrame
data = group1 + group2 + group3
groups = ['A'] * len(group1) + ['B'] * len(group2) + ['C'] * len(group3)

df = pd.DataFrame({
    'score': data,
    'group': groups
})

# ----------------------------
# One-way ANOVA using statsmodels
# ----------------------------
model = ols('score ~ C(group)', data=df).fit()
anova_table = sm.stats.anova_lm(model, typ=2)

print("\nANOVA Results:")
print(anova_table)

# ----------------------------
# Tukey's HSD Post-hoc Test
# ----------------------------
tukey_result = pairwise_tukeyhsd(endog=df['score'],
                                 groups=df['group'],
                                 alpha=0.05)

print("\nTukey's HSD Test Results:")
print(tukey_result.summary())
```

Output:

```
ANOVA Results:
               sum_sq    df           F        PR(>F)
C(group)  476.133333   2.0  82.091954  9.983666e-08
Residual   34.800000  12.0         NaN           NaN

Tukey's HSD Test Results:
Multiple Comparison of Means - Tukey HSD, FWER=0.05
=====================================================
group1 group2 meandiff p-adj   lower   upper  reject
-----------------------------------------------------
     A      B      6.8 0.0001  3.9266  9.6734   True
     A      C     13.8    0.0 10.9266 16.6734   True
     B      C      7.0 0.0001  4.1266  9.8734   True
-----------------------------------------------------
```

Practical: 5

Regression and Its Types

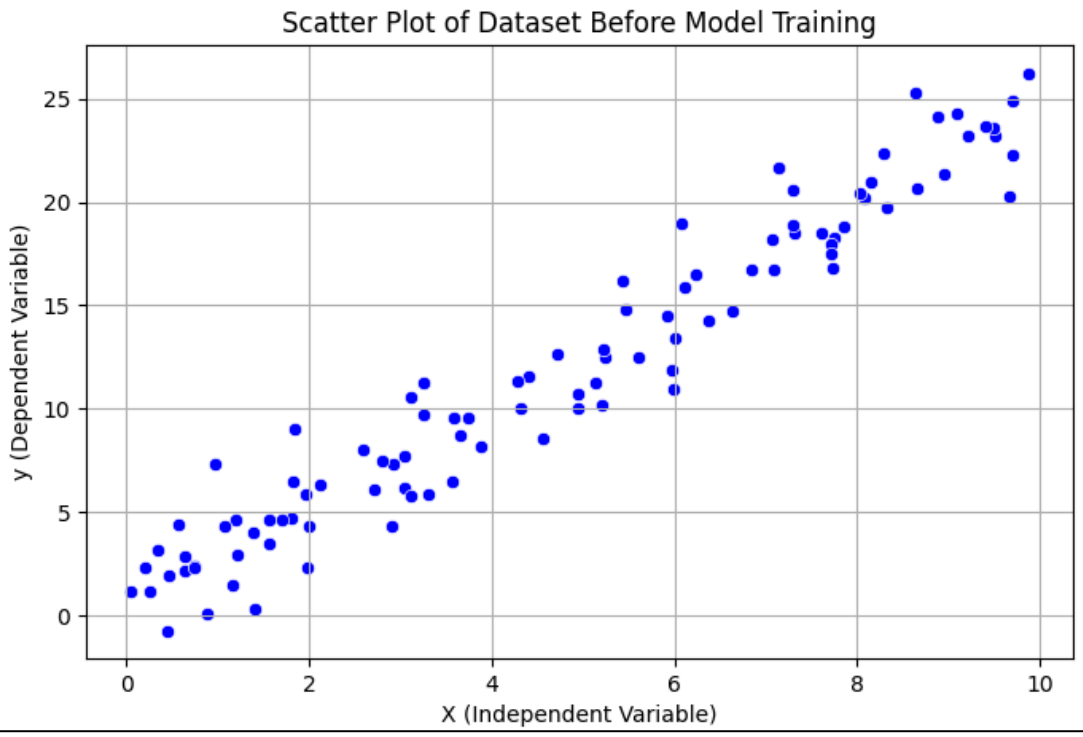
Implement simple linear regression using a dataset.

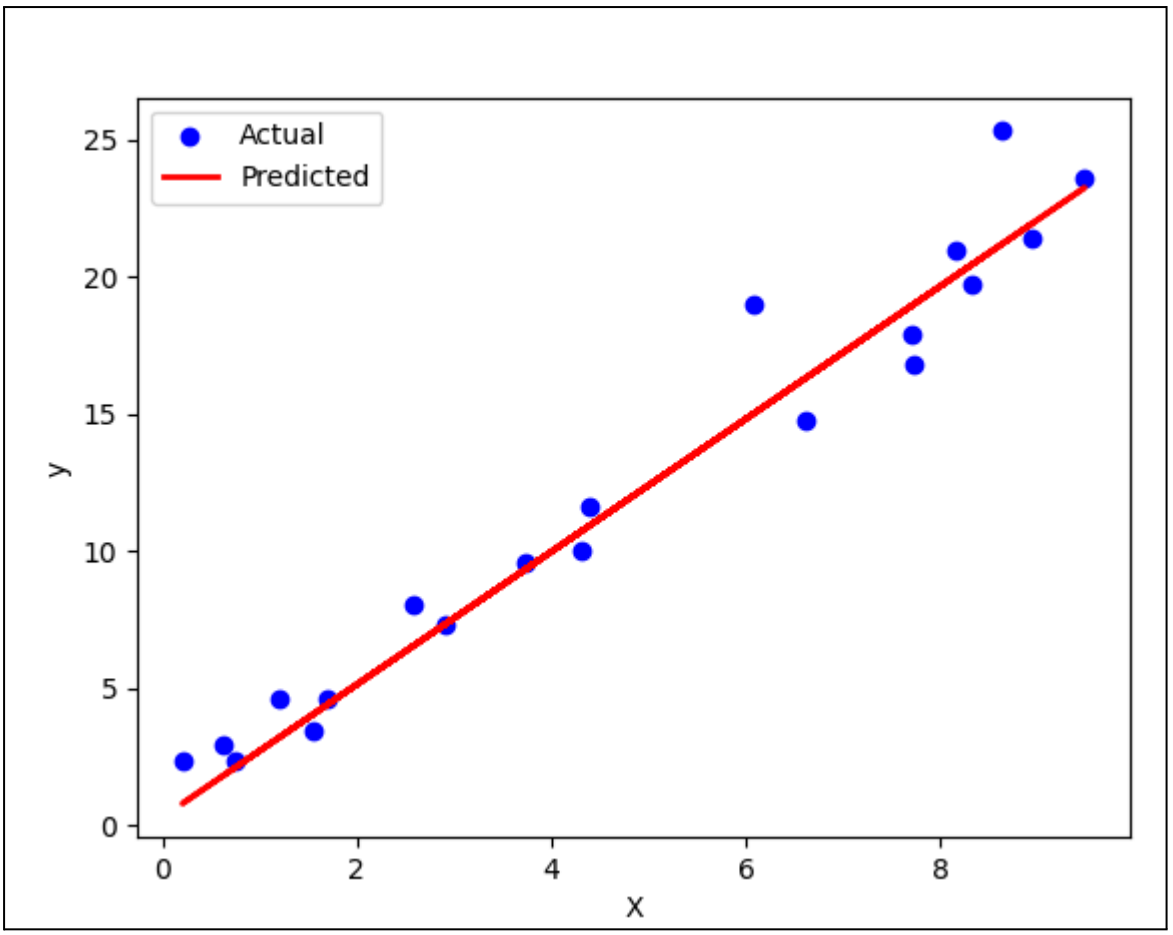
Source Code:

```
# Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Generate synthetic dataset
np.random.seed(42)
X = np.random.rand(100, 1) * 10
y = 2.5 * X + np.random.randn(100, 1) * 2

# Convert to DataFrame
df = pd.DataFrame({'X': X.flatten(), 'y': y.flatten()})
print(df)

# Visualize the Data Before Model Training
plt.figure(figsize=(8, 5))
sns.scatterplot(x=df['X'], y=df['y'], color='blue')
plt.xlabel("X (Independent Variable)")
plt.ylabel("y (Dependent Variable)")
plt.title("Scatter Plot of Dataset Before Model Training")
plt.grid(True)
plt.show()

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train Simple Linear Regression Model
model = LinearRegression()
model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)

# Model Coefficients
print(f"Intercept: {model.intercept_[0]:.4f}, Coefficient: {model.coef_[0][0]:.4f}")

# Model Performance
print(f"Mean Squared Error: {mean_squared_error(y_test, y_pred):.4f}")
print(f"R-squared: {r2_score(y_test, y_pred):.4f}")

# Plot Regression Line
```

```
plt.scatter(X_test, y_test, color='blue', label="Actual")
plt.plot(X_test, y_pred, color='red', linewidth=2, label="Predicted")
plt.xlabel("X")
plt.ylabel("y")
plt.legend()
plt.show()

# Multiple Linear Regression (Adding another predictor)
X_multi = np.hstack((X, np.random.rand(100, 1) * 5))
X_train, X_test, y_train, y_test = train_test_split(X_multi, y, test_size=0.2,
random_state=42)

model_multi = LinearRegression()
model_multi.fit(X_train, y_train)
y_pred_multi = model_multi.predict(X_test)

print(f"\nMultiple Regression Coefficients: {model_multi.coef_}")
print(f"Multiple Regression R-squared: {r2_score(y_test, y_pred_multi):.4f}")
```

Output:






```
           x          y
0   3.745401   9.537597
1   9.507143  23.169843
2   7.319939  18.483370
3   5.986585  10.991324
4   1.560186   3.461122
..       ...        ...
95  4.937956  10.713269
96  5.227328  12.914117
97  4.275410  11.370829
98  0.254191   1.188860
99  1.078914   4.351652

[100 rows x 2 columns]
Intercept: 0.2858, Coefficient: 2.4197
Mean Squared Error: 2.6148
R-squared: 0.9546

Multiple Regression Coefficients: [[2.41775741 0.07570753]]
Multiple Regression R-squared: 0.9558
```

Practical: 6

Logistic Regression and Decision Tree

Build a logistic regression model to predict a binary outcome.

Source Code:

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score,
classification_report

# Load Iris dataset
iris = load_iris()

# Convert to DataFrame
iris_df = pd.DataFrame(
    data=np.c_[iris['data'], iris['target']],
    columns=iris['feature_names'] + ['target']
)

# Create binary classification dataset (remove class 2)
binary_df = iris_df[iris_df['target'] != 2]

# Features and target
X = binary_df.drop('target', axis=1)
y = binary_df['target']

# Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# ----------------------------
# Logistic Regression
# ----------------------------
logistic_model = LogisticRegression()
logistic_model.fit(X_train, y_train)

y_pred_logistic = logistic_model.predict(X_test)

print("\nLogistic Regression Metrics")
print("Accuracy:", accuracy_score(y_test, y_pred_logistic))
print("Precision:", precision_score(y_test, y_pred_logistic))
print("Recall:", recall_score(y_test, y_pred_logistic))
print("\nClassification Report:\n", classification_report(y_test, y_pred_logistic))

# ----------------------------
# Decision Tree Classifier
# ----------------------------
```

```
decision_tree_model = DecisionTreeClassifier()
decision_tree_model.fit(X_train, y_train)

y_pred_tree = decision_tree_model.predict(X_test)

print("\nDecision Tree Metrics")
print("Accuracy:", accuracy_score(y_test, y_pred_tree))
print("Precision:", precision_score(y_test, y_pred_tree))
print("Recall:", recall_score(y_test, y_pred_tree))
print("\nClassification Report:\n", classification_report(y_test, y_pred_tree))
```

Output:

```
Logistic Regression Metrics
Accuracy: 1.0
Precision: 1.0
Recall: 1.0

Classification Report:
               precision    recall  f1-score   support

         0.0       1.00      1.00      1.00        12
         1.0       1.00      1.00      1.00         8

    accuracy                           1.00        20
   macro avg       1.00      1.00      1.00        20
weighted avg       1.00      1.00      1.00        20


Decision Tree Metrics
Accuracy: 1.0
Precision: 1.0
Recall: 1.0

Classification Report:
               precision    recall  f1-score   support

         0.0       1.00      1.00      1.00        12
         1.0       1.00      1.00      1.00         8

    accuracy                           1.00        20
   macro avg       1.00      1.00      1.00        20
weighted avg       1.00      1.00      1.00        20
```

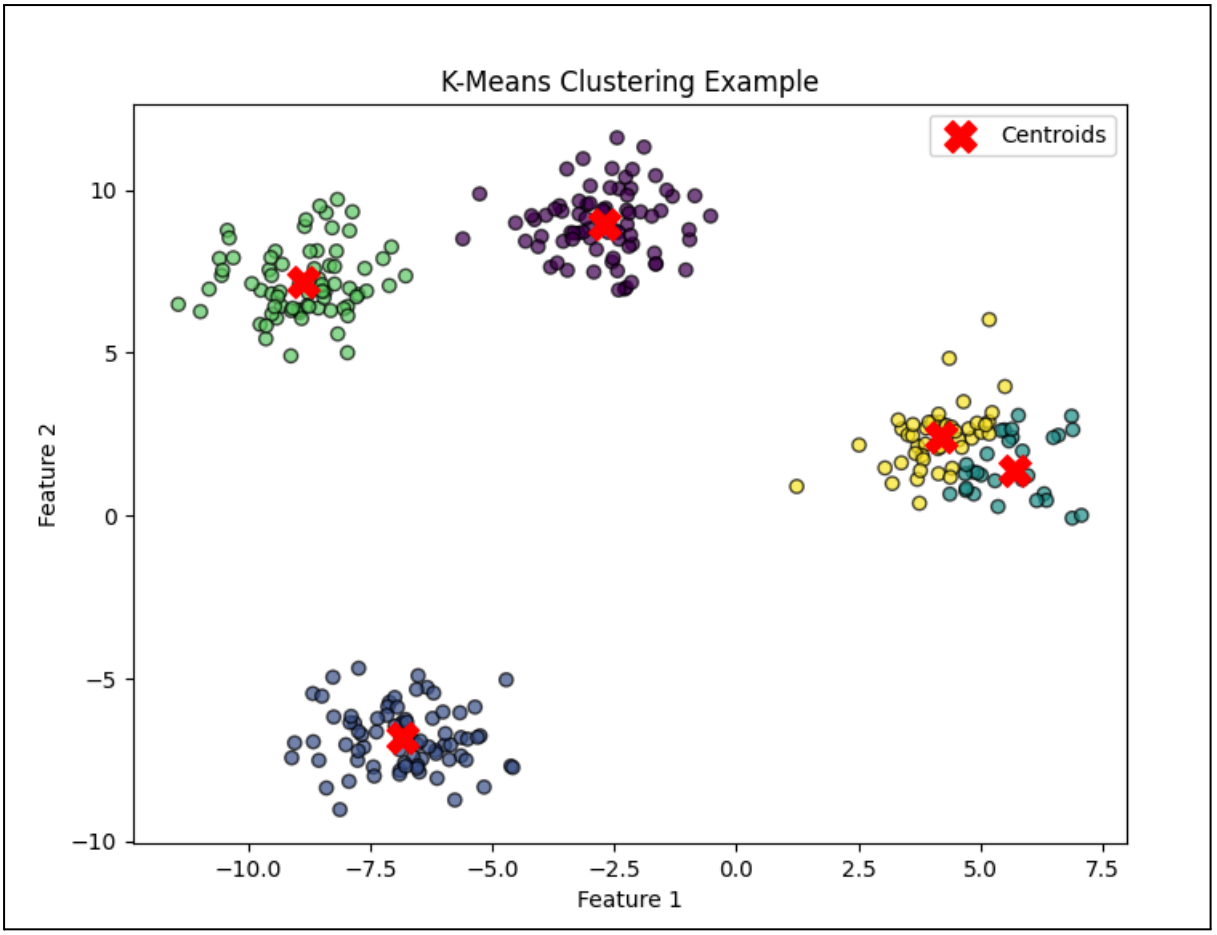
Practical: 7

K-Means Clustering

Source Code:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

# Generate sample data
X, y = make_blobs(n_samples=300, centers=4, cluster_std=1.05, random_state=42)

# Perform K-Means clustering
kmeans = KMeans(n_clusters=5, random_state=42)
kmeans.fit(X)
y_kmeans = kmeans.predict(X)

# Plot results
plt.figure(figsize=(8, 6))
plt.scatter(X[:, 0], X[:, 1], c=y_kmeans, cmap='viridis', marker='o', edgecolors='k',
alpha=0.7)
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1], s=200, c='red',
marker='X', label='Centroids')

plt.title("K-Means Clustering Example")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.legend()
plt.show()
```

Output:


K-Means Clustering Example
Centroids
Feature 2
Feature 1
10
5
0
−5
−10
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5

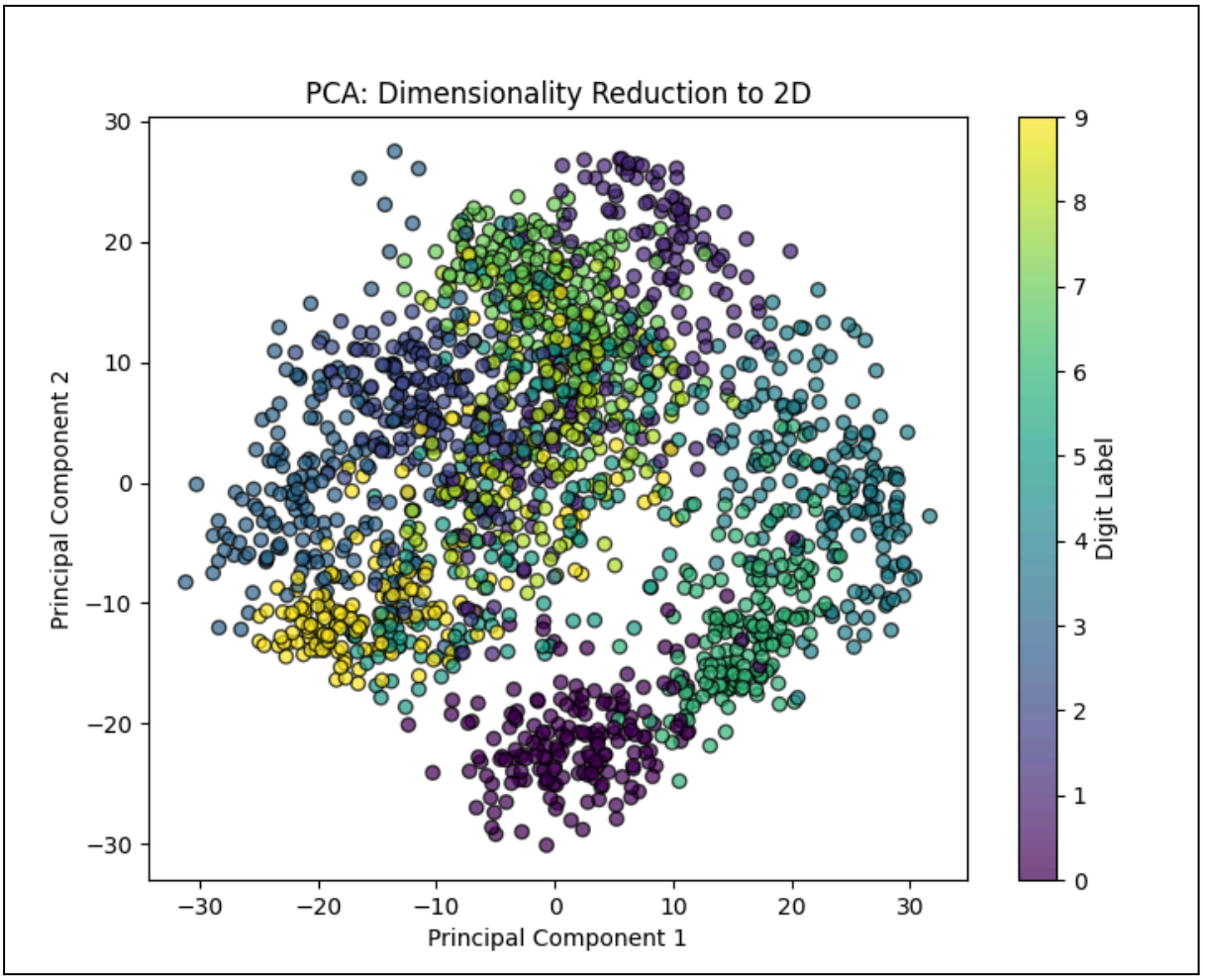
Practical: 8

Principal Component Analysis (PCA)

Source Code:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.datasets import load_digits

# Load dataset (Handwritten digits)
digits = load_digits()
X = digits.data  # Feature matrix (64 features per sample)

# Perform PCA to reduce dimensions to 2
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

# Scatter plot of transformed data
plt.figure(figsize=(8, 6))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=digits.target, cmap='viridis', edgecolors='k',
alpha=0.7)
plt.colorbar(label="Digit Label")
plt.title("PCA: Dimensionality Reduction to 2D")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.show()

# Explained variance ratio
print(f"Explained variance ratio: {pca.explained_variance_ratio_}")
print(f"Total variance explained: {sum(pca.explained_variance_ratio_)*100:.2f}%")
```

Output:


PCA: Dimensionality Reduction to 2D
Principal Component 2
Principal Component 1
Digit Label
30
20
10
0
−10
−20
−30
−30
−20
−10
0
10
20
30
9
8
7
6
5
4
3
2
1
0

Practical: 9

Data Visualization

Source Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Set Seaborn style
sns.set(style="whitegrid")

# Load Dataset (Titanic dataset for example)
df = sns.load_dataset("titanic")

# Display first few rows
print(df.head())

# Check missing values
print(df.isnull().sum())

# Summary statistics
print(df.describe())

# Count of passengers by class
plt.figure(figsize=(8, 5))
sns.countplot(x="class", data=df, palette="Set2")
plt.title("Number of Passengers by Class")
plt.xlabel("Passenger Class")
plt.ylabel("Count")
plt.show()
# Survival count by class
```

```
plt.figure(figsize=(8, 5))
sns.barplot(x="class", y="survived", data=df, palette="coolwarm")
plt.title("Survival Rate by Class")
plt.xlabel("Passenger Class")
plt.ylabel("Survival Rate")
plt.show()

plt.figure(figsize=(8, 5))
sns.barplot(x="sex", y="survived", data=df, palette="viridis")
plt.title("Survival Rate by Gender")
plt.xlabel("Gender")
plt.ylabel("Survival Rate")
plt.show()

plt.figure(figsize=(10, 5))
sns.histplot(df[df["survived"] == 1]["age"], bins=30, kde=True, color="green",
label="Survived")
sns.histplot(df[df["survived"] == 0]["age"], bins=30, kde=True, color="red", label="Not
Survived")
plt.legend()
plt.title("Age Distribution of Titanic Survivors vs Non-Survivors")
plt.xlabel("Age")
plt.ylabel("Count")
plt.show()
```

Output:

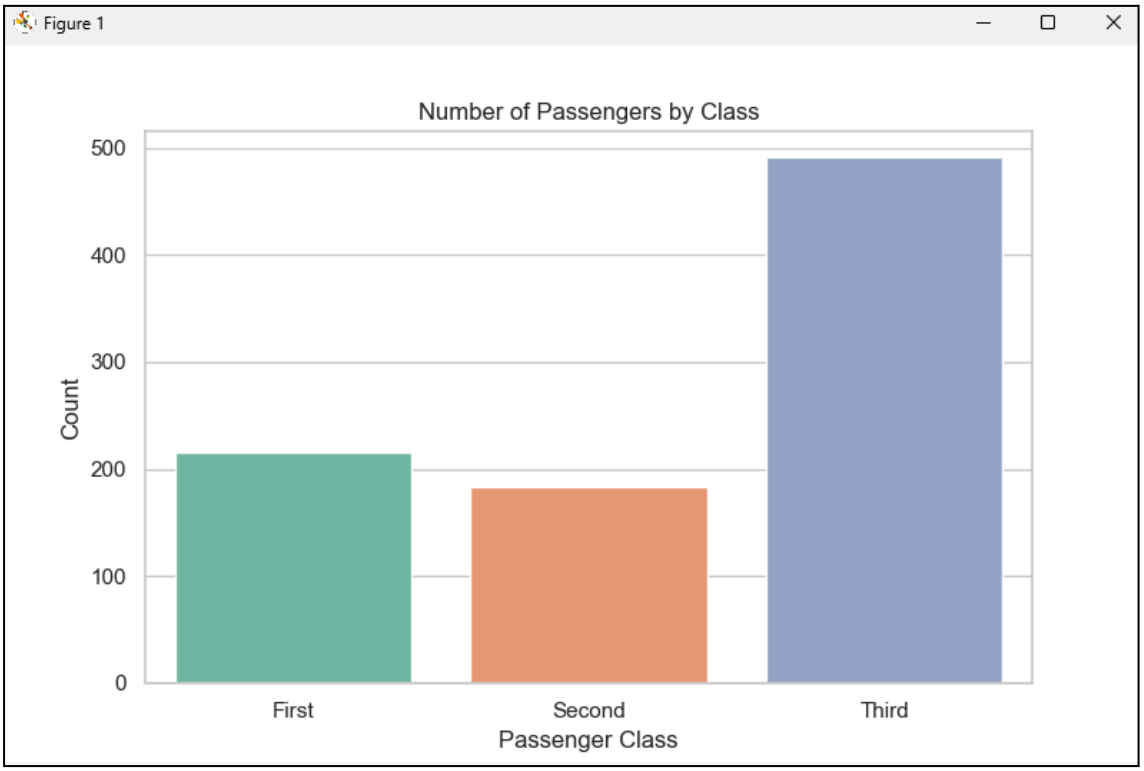


Figure 1
Number of Passengers by Class
Count
500
400
300
200
100
0
First
Second
Third
Passenger Class

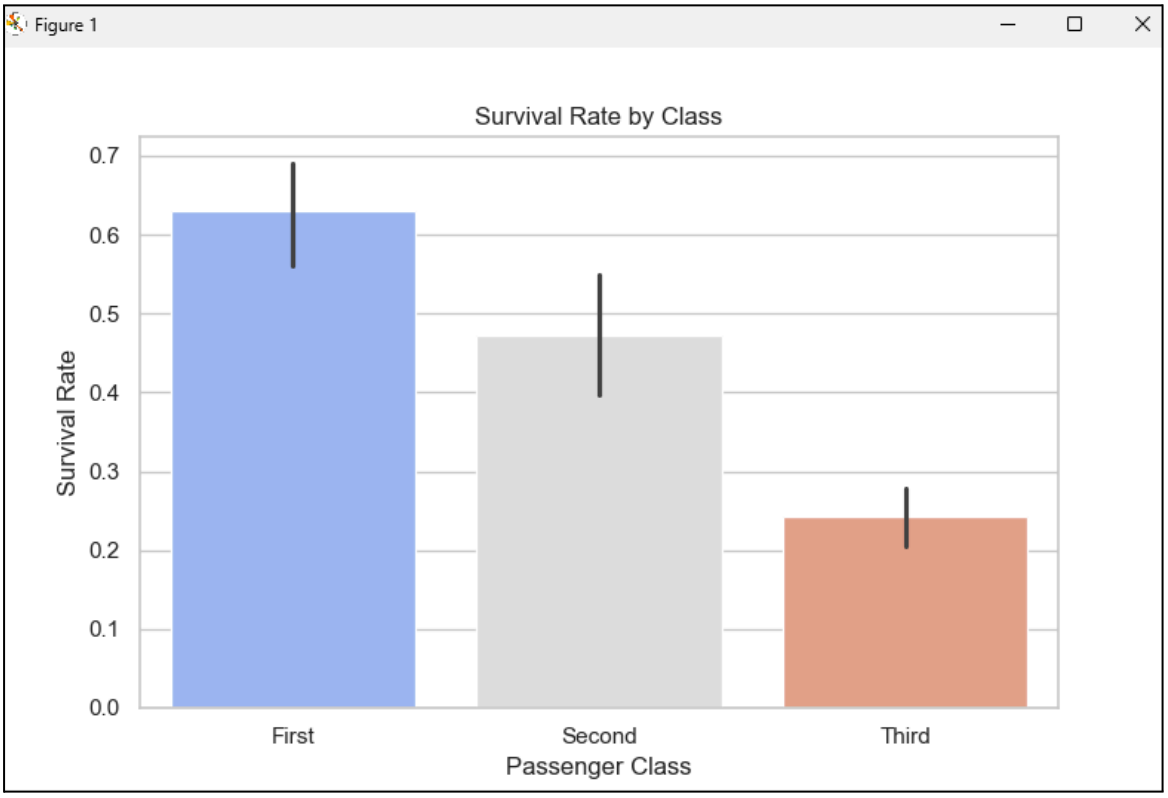
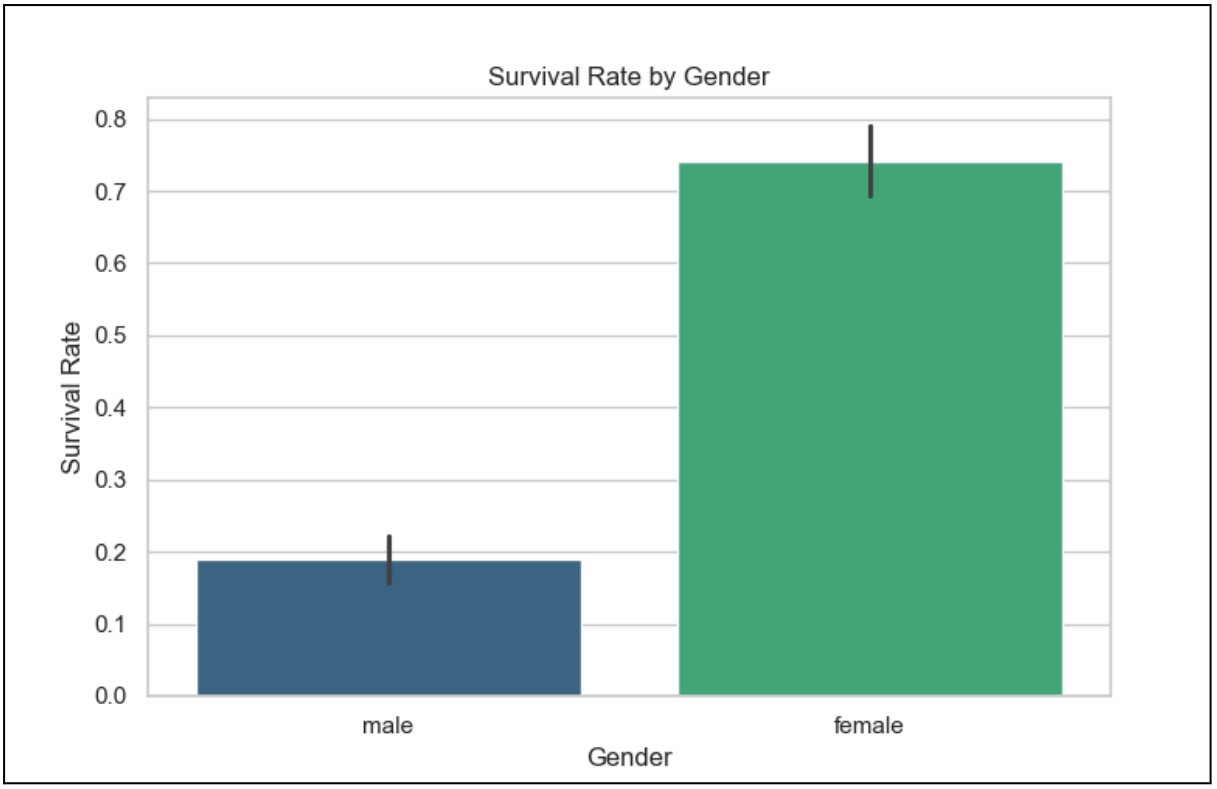


Figure 1
Survival Rate by Class
Survival Rate
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0.0
First
Second
Third
Passenger Class


Survival Rate by Gender
Survival Rate
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
male
female
Gender

Age Distribution of Titanic Survivors vs Non-Survivors
Survived
Not Survived
Count
0
10
20
30
40
0
10
20
30
40
50
60
70
80

```
   survived  pclass     sex   age  ...  deck  embark_town  alive  alone
0         0       3    male  22.0  ...   NaN  Southampton     no  False
1         1       1  female  38.0  ...     C    Cherbourg    yes  False
2         1       3  female  26.0  ...   NaN  Southampton    yes   True
3         1       1  female  35.0  ...     C  Southampton    yes  False
4         0       3    male  35.0  ...   NaN  Southampton     no   True

[5 rows x 15 columns]
survived         0
pclass           0
sex              0
age            177
sibsp            0
parch            0
fare             0
embarked         2
class            0
who              0
adult_male       0
deck           688
embark_town      2
alive            0
alone            0
dtype: int64
         survived      pclass         age       sibsp       parch        fare
count  891.000000  891.000000  714.000000  891.000000  891.000000  891.000000
mean     0.383838    2.308642   29.699118    0.523008    0.381594   32.204208
std      0.486592    0.836071   14.526497    1.102743    0.806057   49.693429
min      0.000000    1.000000    0.420000    0.000000    0.000000    0.000000
25%      0.000000    2.000000   20.125000    0.000000    0.000000    7.910400
50%      0.000000    3.000000   28.000000    0.000000    0.000000   14.454200
75%      1.000000    3.000000   38.000000    1.000000    0.000000   31.000000
max      1.000000    3.000000   80.000000    8.000000    6.000000  512.329200
```